

ระบบยืนยันตัวตนด้วยใบหน้าแบบ client-server พร้อมชั้นป้องกันการปลอมแปลงและหน้ากาก 3 มิติ

eKYC Server — โทรศัพท์เก็บหลักฐาน เซิร์ฟเวอร์ตัดสินใจ · ทดสอบบน Railway

ชื่อระบบ	eKYC Server (<code>server/</code>) และโมดูล <code>@ekyc/react-native-ekyc</code> พร้อมแอปตัวอย่าง <code>apps/example</code>
เวอร์ชันเอกสาร	1.0
วันที่จัดทำ	19 สิงหาคม 2569
สถานะ	ฉบับส่งมอบเพื่อทดสอบภายใน (ไม่ใช่เอกสารรับรองมาตรฐาน)
เซิร์ฟเวอร์ทดสอบ	https://ekyc-api-production-1c11.up.railway.app
แหล่งโค้ด	https://github.com/watcharaponthod-code/ekyc
ไฟล์ติดตั้ง	GitHub Releases ของโครงการ (ไฟล์ <code>ekyc-example-*.apk</code>)

เอกสารฉบับนี้อธิบายสถาปัตยกรรม ชั้นป้องกัน การติดตั้งบน Railway ผลการทดสอบอ้างอิง วิธีประเมินตามตัวชี้วัด ISO/IEC 30107-3 และข้อจำกัดตามสภาพจริง ณ วันที่จัดทำ ระบบนี้ยังไม่ผ่านการประเมินโดยห้องปฏิบัติการภายนอก และไม่ควรรออ้างว่าเป็นไปตามมาตรฐาน ISO/IEC 30107-3

สารบัญ

1. บทนำและวัตถุประสงค์

2. สถาปัตยกรรม

3. ชั้นป้องกันและภัยคุกคามที่รับมือ

3.1 ชั้นเดิม

3.2 ชั้นที่เพิ่มสำหรับหน้ากาก 3 มิติ / ซิลิโคน

4. การติดตั้งบน Railway

5. ส่วนต่อประสาน (API)

6. การทดสอบและรูปร่าง

6.1 การทดสอบอัตโนมัติ

6.2 การตรวจสอบเซิร์ฟเวอร์ที่ติดตั้งจริง

6.3 การเปรียบเทียบ rPPG บนสัญญาณสังเคราะห์

6.4 หน้าจอของแอปตัวอย่าง

7. การประเมินตามตัวชี้วัด ISO/IEC 30107-3

8. การใช้งานแอปตัวอย่างกับเซิร์ฟเวอร์ทดสอบ

9. ข้อจำกัดและแผนงานถัดไป

ภาคผนวก ก. รหัสเหตุผลจากเซิร์ฟเวอร์

ภาคผนวก ข. ตัวแปรสภาพแวดล้อมที่สำคัญ

1. บทนำและวัตถุประสงค์

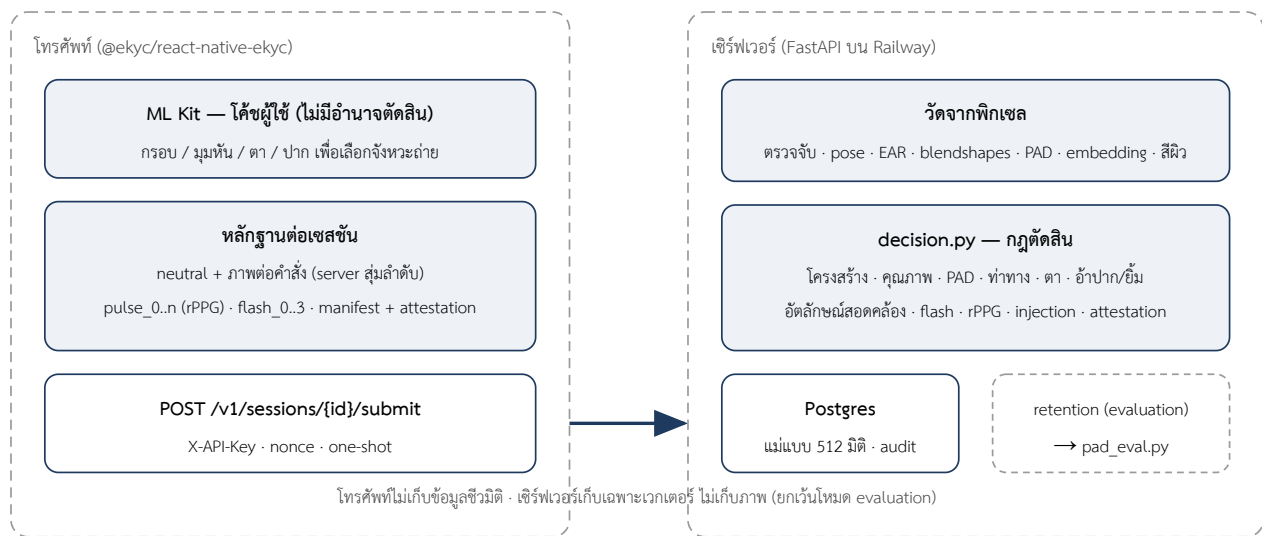
ระบบ eKYC Server เป็นระบบยืนยันตัวตนด้วยใบหน้าที่แบ่งหน้าที่ชัดเจน: **โทรศัพท์** ทำหน้าที่โค้ชผู้ใช้ เก็บหลักฐาน (ภาพนิ่งต่อคำสั่ง ภาพช่วง flash สี และภาพต่อเนื่องช่วงวัดชีพจร) และส่งขึ้นเซิร์ฟเวอร์ ส่วน **เซิร์ฟเวอร์** เป็นผู้ตัดสินใจ แต่เพียงผู้เดียว โดยวัดทุกอย่างซ้ำจากฟิสิกส์ ไม่เชื่อคำใด ๆ ที่โทรศัพท์รายงาน เพราะอุปกรณ์อยู่ในมือของผู้ที่อาจเป็นผู้โจมตี

เอกสารฉบับนี้ครอบคลุมงานรอบวันที่ 18 สิงหาคม 2569 ซึ่งเพิ่ม (1) ชั้นป้องกันหน้ากาก 3 มิติและซิลิโคน (2) เครื่องมือประเมินตามตัวชี้วัด ISO/IEC 30107-3 (3) การติดตั้งเซิร์ฟเวอร์ทดสอบบน Railway พร้อมกุญแจ API และ (4) การปรับแอปตัวอย่างให้ใช้กับเซิร์ฟเวอร์ดังกล่าว

วัตถุประสงค์

1. ให้บริการลงทะเบียน (enroll) ยืนยัน 1:1 (verify) และค้นหา 1:N (identify) ด้วยใบหน้า โดยมีการตรวจสอบความมีชีวิตที่เซิร์ฟเวอร์ตรวจได้จริงทุกชั้น
2. รับมือการโจมตีด้วยภาพถ่าย จอภาพ วิดีโอย้อนหลัง การสลับบุคคล การเล่นซ้ำหลักฐาน และ—ในรอบนี้— หน้ากากแข็งและหน้ากากซิลิโคน
3. ให้ทีมวัดสมรรถนะด้วยตัวชี้วัดเดียวกับห้องปฏิบัติการมาตรฐาน ก่อนตัดสินใจส่งประเมินภายนอก

2. สถาปัตยกรรม



รูปที่ 1 การแบ่งหน้าที่ระหว่างโทรศัพท์และเซิร์ฟเวอร์

ตารางที่ 1 องค์ประกอบ

ส่วน	เทคโนโลยี
โมดูลโทรศัพท์	React Native, react-native-vision-camera, ML Kit (face detector), <code>@ekyc/react-native-ekyc</code>
เซิร์ฟเวอร์	Python 3.12, FastAPI, SQLAlchemy; backend การมองเห็นเลือกได้: <code>deepface</code> (MediaPipe Face Landmarker + DeepFace ArcFace + MiniFASNet ensemble) หรือ <code>onnx</code> (SCRFD + ArcFace + MiniFASNet ผ่าน onnxruntime — ใช้บน Railway)
ฐานข้อมูล	PostgreSQL (Railway) — บุคคล แม่แบบ เซสชัน บันทึกตรวจสอบ
การติดตั้ง	Docker image (python:3.12-slim + onnxruntime, ดึงน้ำหนักโมเดลตอน build) บน Railway

3. ชั้นป้องกันและภัยคุกคามที่รับมือ

3.1 ชั้นเดิม

ตารางที่ 2 ชั้นป้องกันเดิมและภัยคุกคามที่จับได้

ชั้น	ภัยคุกคาม
MiniFASNet PAD (ทั้งเฟรม + กรอบใบหน้า)	ภาพถ่าย จอภาพ วิดีโอย้อนหลัง (AUC 1.000 บน screen replay ที่วัดไว้, n=70)
คำสั่งสุ่มลำดับโดยเซิร์ฟเวอร์ + ตรวจ pose/EAR ซ้ำจากฟิสิกเซล	วิดีโอที่อัดไว้ล่วงหน้า
Flash สุ่มจากหน้าจอ ('flash.py')	ภาพถ่าย/จอ/สตรีมที่ฉีดเข้ามา — ผิดหวังจริงต้องสะท้อนสีตามลำดับที่สุ่ม
Nonce + TTL 120 วินาที + one-shot + hash ของเฟรม	เล่นซ้ำหลักฐานเดิม, สตรีมนิ่ง
ความสอดคล้องของอัตลักษณ์ (ArcFace ทุกคู่ภาพ)	ผ่าน liveness ด้วยคน A แล้วส่งภาพคน B
Planarity (advisory)	ภาพแบน/ข้อมูลผิดปกติ (จับภาพถ่ายไม่ได้ ตามที่วัดไว้)

3.2 ชั้นที่เพิ่มสำหรับหน้ากาก 3 มิติ / ซิลิโคน

คนที่สวมหน้ากากซิลิโคนหันหัวได้ กระพริบตาได้ (ตาจริงอยู่หลังรู) และสะท้อนแสง flash ได้เหมือนผิวหนัง ชั้นเดิมทั้งหมดจึงไม่ได้ออกแบบมาจับกรณีนี้ รอบนี้เพิ่มสองชั้นที่ตอบสนองภัยทางกายภาพที่หน้ากากไม่มี

ตารางที่ 3 ชั้นป้องกันใหม่

ชั้น	วิธีการ	จับอะไร	สถานะ
------	---------	---------	-------

คำสั่งอัปปาก / ยิ้ม ที่ เซิร์ฟเวอร์ตรวจได้	วัด <code>jawOpen</code> / <code>mouthSmile*</code> จาก MediaPipe blendshapes บนเฟรมคำสั่ง เทียบเฟรม neutral ของบุคคลเดียวกัน ต้อง ผ่านทั้งค่าสมบูรณ์ (≥ 0.35 / 0.45) และค่า เปลี่ยนแปลง (≥ 0.20 / 0.25); เซิร์ฟเวอร์ออก คำสั่งนี้เฉพาะเมื่อ backend วัดได้ และ full tier บังคับมีอัปปากทุกครั้งในตำแหน่งสุม	หน้ากากแข็ง (3D-print, resin, กระดาษ, latex) ขากรรไกรอ้าไม่ได้; ซิลิโคนยืดหยุ่นจับได้บางส่วน	บังคับใช้ (enforce)
rPPG pulse liveness	หลังคำสั่งครบ ผู้ใช้ยูนิง 7-8 วินาที แอปถ่าย ภาพต่อเนื่อง เซิร์ฟเวอร์อ่านสีเฉลี่ยของหน้าผาก และแก้มสองข้าง (MediaPipe landmarks) ต่อ เฟรม $\rightarrow$ POS projection $\rightarrow$ สเปกตรัมช่วง 0.7-3 Hz เฉลี่ยข้าม 3 patch $\rightarrow$ ความเด่น ของยอด (dB) $\rightarrow$ คะแนน	ซิลิโคน / latex — ไม่มี เลือดไหลเวียน ไม่มี ซีพจร	advisory (บันทึกคะแนน ไม่ตัดสิน) จนกว่าจะวัดบน โทรศัพท์จริง
Identity anchor ของ pulse	เฟรมแรก/สุดท้ายของช่วง pulse ถูกฝังและรวม เข้าการตรวจอัตลักษณ์สอดคล้อง	บุคคลจริงมาถ่ายช่วง pulse แทนผู้สวม หน้ากาก	บังคับใช้
ขอบเขต PAD	ค่า PAD ต่ำสุดคิดเฉพาะเฟรมคำสั่ง (เฟรม flash สีจัดถูกตัดออก)	ลด false reject	บังคับใช้
กุญแจ API	<code>X-API-Key</code> หรือ <code>Authorization: Bearer</code> ทุกเส้นทางยกเว้น <code>/v1/health</code>	API เปิดสาธารณะ	บังคับใช้บน Railway
Attestation hook	แอปส่ง token Play Integrity / App Attest ในกล่องหลักฐาน; เซิร์ฟเวอร์บังคับได้ด้วย <code>EKYC_REQUIRE_ATTESTATION</code>	สตรีมที่ติดบนเครื่อง root	ตรวจการมีอยู่ เท่านั้น ยังไม่ verify กับ Google/Apple

ข้อจำกัดสำคัญ: คำสั่งอัปปาก/ยิ้มต้องใช้ backend `deepface` (MediaPipe blendshapes) เซิร์ฟเวอร์ ทดสอบบน Railway ใช้ backend `onnx` จึงไม่ออกคำสั่งนี้โดยอัตโนมัติ; rPPG ทำงานได้กับทุก backend แต่ บน `onnx` ใช้ patch เดียว (กลางกรอบใบหน้า) ซึ่งอ่อนกว่า

4. การติดตั้งบน Railway

ตารางที่ 4 ข้อมูลเซิร์ฟเวอร์ทดสอบ

URL	<code>https://ekyc-api-production-1c11.up.railway.app</code>
โครงการ / บริการ	Railway project <code>ekyc-server</code> → service <code>ekyc-api</code> + <code>Postgres</code>
Builder	Dockerfile (<code>server/Dockerfile</code>) — python:3.12-slim, onnxruntime, ดึงน้ำหนักโมเดล ONNX (~200 MB, ตรวจสอบ SHA-256) ตอน build
Backend	<code>onnx</code> ; ตั้ง <code>EKYC_NEUTRAL_YAW_MAX_DEG=45</code> ตามที่ต้องใช้กับ backend นี้
Flash / Pulse	flash เปิด 4 เฟรม; pulse ปิด (เปิดได้ด้วย <code>EKYC_PULSE_FRAMES=90</code> , <code>EKYC_PULSE_DURATION_MS=8000</code>)
กุญแจ API	อยู่ใน Railway → Variables → <code>EKYC_API_KEYS</code> เท่านั้น ไม่อยู่ในซอร์สโค้ด หมุนเปลี่ยนได้ที่นั้น
ฐานข้อมูล	<code>DATABASE_URL</code> อ้างอิงบริการ Postgres อัตโนมัติ; ตารางสร้างตอนเริ่มระบบ
Healthcheck	<code>/v1/health</code> (เปิดโดยไม่ต้องใช้กุญแจ)
Redeploy	<code>cd server && railway up --service ekyc-api --detach</code>

บันทึกปัญหาที่พบระหว่างติดตั้ง

การ deploy ครั้งแรกล้มเหลวที่ขั้น healthcheck เนื่องจาก `startCommand` ใน `railway.json` ระบุ `--port $PORT` ซึ่ง Railway เรียกใช้โดยไม่ผ่าน shell ค่า `$PORT` จึงไม่ถูกแทนที่ (uvicorn รายงาน “'\$PORT' is not a valid integer”) แก้ไขโดยลบ `startCommand` ออก และใช้ `CMD ["sh", "-c", "uvicorn ... --port ${PORT:-8000}"]` ใน Dockerfile แทน การ deploy ครั้งที่สองผ่าน healthcheck และให้บริการตามปกติ

5. ส่วนต่อประสาน (API)

ตารางที่ 5 เส้นทาง API เวอร์ชัน 1

เส้นทาง	หน้าที่
GET /v1/health	สถานะ, backend, โมเดลที่โหลด (ไม่ต้องใช้กุญแจ)
POST /v1/sessions	สร้างเซสชัน: {purpose: enroll verify identify, personId?, displayName?, tier?, label?} → sessionId, nonce, ลำดับคำสั่ง, แผน flash, แผน pulse, นโยบายเวลา
POST /v1/sessions/{id}/submit	multipart: manifest (JSON) + frames (ไฟล์ neutral.jpg, <challenge>.jpg, flash_i.jpg, pulse_i.jpg) → {decision, reasons, scores, personId?, match?}
GET /v1/persons	รายชื่อบุคคลที่ลงทะเบียน
DELETE /v1/persons/{id}	ลบบุคคลและแม่แบบ
POST /v1/client-log	บรรทัดวินิจฉัยจากโทรศัพท์

ทุกเส้นทางยกเว้น health ต้องส่ง X-API-Key: <key> มิฉะนั้นได้ 401 (ไม่มีกุญแจ) หรือ 403 (กุญแจผิด)

6. การทดสอบและรูปร่าง

6.1 การทดสอบอัตโนมัติ

ตารางที่ 6 ผลชุดทดสอบอัตโนมัติ (18 ส.ค. 2569)

ชุดทดสอบ	ผล	ครอบคลุม
เซิร์ฟเวอร์ — fast suite (pytest, ไม่ต้องใช้โมเดล)	151 ผ่าน / 151 (เดิม 84)	กฎตัดสินทุกข้อ รวมอ้าปาก/ยิ้ม, PAD scope, rPPG (scorer + decision + API), ฤๅญแจ API, retention, pad_eval, flash, injection, planarity, API end-to-end ด้วย backend จำลอง
เซิร์ฟเวอร์ — model suite (pytest -m models)	42 รายการ (ต้องมีน้ำหนักโมเดล; ไม่ได้รันในรอบนี้)	MediaPipe/DeepFace/ONNX บนภาพจริง, end-to-end enroll → verify → identify
โมดูลโทรศัพท์ (jest)	105 ผ่าน / 105 (เดิม 96)	LivenessSession, ทุก Challenge, mouthOpenness, ข้อความทักท้วง, EKYCClient (ฤๅญแจ API)

6.2 การตรวจสอบเซิร์ฟเวอร์ที่ติดตั้งจริง

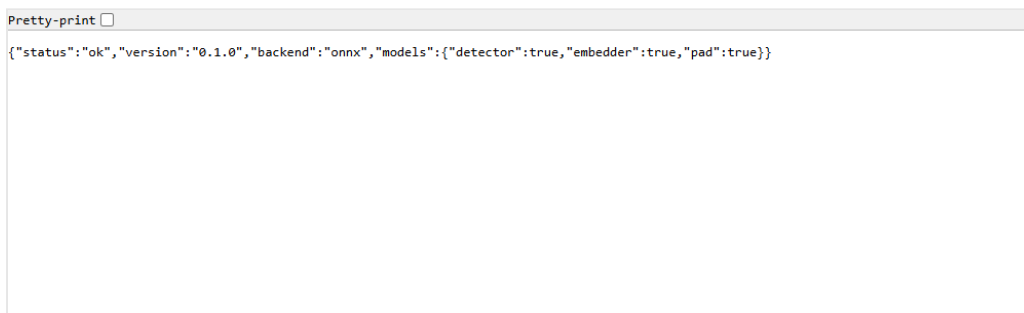
ผลการเรียกใช้จริงหลัง deploy (18 ส.ค. 2569 เวลา 15:20 น. ตามเวลาไทย):

```
$ curl https://ekyc-api-production-1c11.up.railway.app/v1/health
{"status":"ok","version":"0.1.0","backend":"onnx","models":{"detector":true,"embedder":true,"pad":true}}

$ curl -o /dev/null -w "%{http_code}" https://ekyc-api-production-1c11.up.railway.app/v1/persons
401                                     ← ไม่มีกุญแจ

$ curl -H "X-API-Key: *****" .../v1/persons
[]

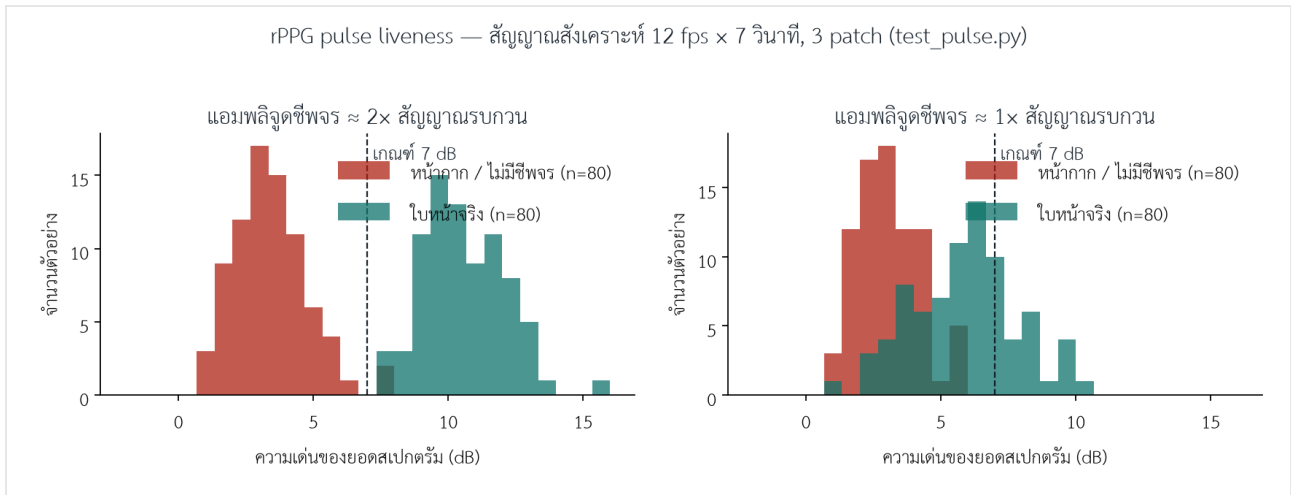
$ curl -H "X-API-Key: *****" -H "Content-Type: application/json" \
  -d '{"purpose":"enroll","displayName":"smoke"}' .../v1/sessions
{"sessionId":"s_39127d65e69f712d2baf0203","nonce":"4Mef...", "challenges":["closeEyes","turnLeft","turnRight"],
  "flash":["green","white","red","blue"],"pulse":null,"expiresAt":"2026-08-18T08:22:41Z",
  "policy":{"holdMs":400,"perStepTimeoutMs":12000,"totalTimeoutMs":60000}}
```



รูปที่ 2 การเรียก `/v1/health` ของเซิร์ฟเวอร์ทดสอบผ่านเบราว์เซอร์ (19 ส.ค. 2569)

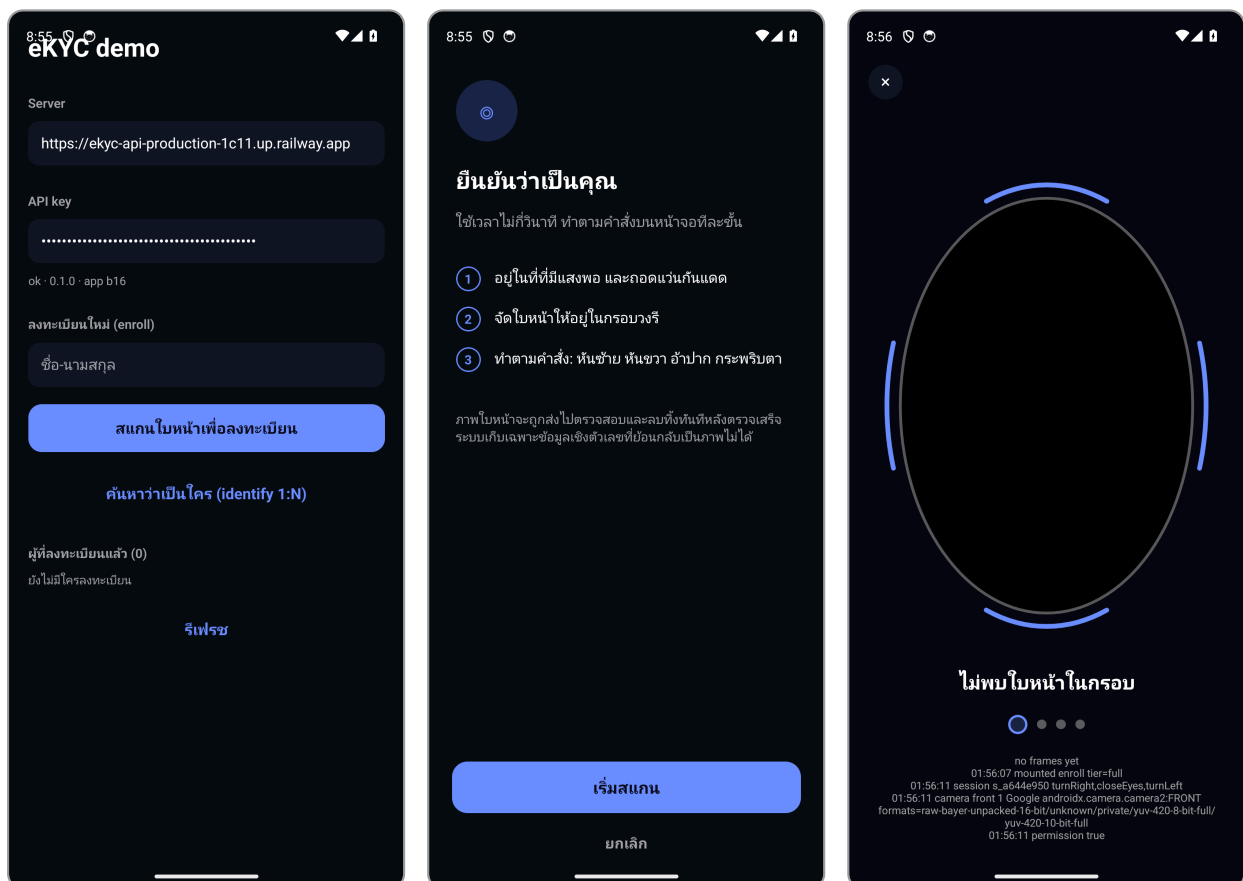
6.3 การเปรียบเทียบ rPPG บนสัญญาณสังเคราะห์

ตัวให้คะแนน rPPG ทดสอบด้วยสัญญาณสีผิวสังเคราะห์ (ambient + แอมพลิจูดชีพจร $\times \sin$ + สัญญาณรบกวนแบบเกาส์ ต่อ patch) ที่อัตราเฟรม 8–30 fps และช่วงเวลา 7 วินาที เทียบกับสัญญาณที่ไม่มีชีพจร (หน้ากาก) ผลแสดงในรูปที่ 3



รูปที่ 3 การกระจายความเด่นของยอดสเปกตรัม: เมื่อแอมพลิจูดชีพจรสูงกว่าสัญญาณรบกวน ≈ 2 เท่า สองกลุ่มแยกจากกันชัดที่เกณฑ์ 7 dB (ซ้าย); เมื่อใกล้เคียงกันจะซ้อนทับ (ขวา) — เหตุผลที่ขั้นนี้เริ่มต้นเป็น advisory

6.4 หน้าจอของแอปตัวอย่าง



รูปที่ 4 แอปตัวอย่าง (apps/example, build 16) บน Android Emulator (API 34): หน้าแรกแสดงการเชื่อมต่อเซิร์ฟเวอร์ Railway สำเร็จ (“ok · 0.1.0”) หลังใส่กุญแจ API, หน้าแนะนำ, หน้าจับภาพซึ่งบันทึกวินิจฉัยแสดงว่าเซิร์ฟเวอร์ออกเซชันและลำดับคำสั่งแล้ว (turnRight, closeEyes, turnLeft) — กล้อง emulator ไม่มีใบหน้าจริงจึงแสดง “ไม่พบใบหน้าในกรอบ”

7. การประเมินตามตัวชี้วัด ISO/IEC 30107-3

ISO/IEC 30107-3 เป็นมาตรฐานว่าด้วย **วิธีทดสอบ** การตรวจจับการโจมตีด้วยสื่อบล็อก การ “ผ่าน” มาตรฐานนี้ หมายถึงห้องปฏิบัติการที่ได้รับการรับรอง (เช่น iBeta, Fime) นำเครื่องมือโจมตี (PAI) จริ่งมาทดสอบตามโปรโตคอลของตนแล้วรายงานผล จึง**ไม่สามารถทำให้ผ่านได้ด้วยโค้ดเพียงอย่างเดียว** สิ่งที่ต้องการจัดทำคือเครื่องมือวัดด้วยตัวชี้วัดเดียวกัน เพื่อให้ตัวเลขที่จะนำไปยื่นมีความน่าเชื่อถือ และพบช่องโหว่ก่อนห้องปฏิบัติการ

ตารางที่ 7 ตัวชี้วัดที่ `scripts/pad_eval.py` รายงาน

ตัวชี้วัด	ความหมาย
APCER (ต่อชนิด PAI)	สัดส่วนการโจมตีที่ถูกจับเป็นของจริง; เมื่อรายงานเป็นค่าเดียวใช้ชนิดที่แย่ที่สุด
BPCER	สัดส่วนการนำเสนอของจริงที่ถูกปฏิเสธ
ACER	$(APCER_{\max} + BPCER) / 2$
APNRR / BPNRR	อัตราไม่ตอบสนอง (ระบบจับภาพไม่ได้/ผิดโปรโตคอล) รายงานแยก ไม่นับรวมใน APCER/BPCER
ช่วงความเชื่อมั่น 95 %	Wilson interval ทุกอัตรา — 0/30 ไม่ใช่ “0 %” แต่คือ “ $\leq$ ประมาณ 11 %”
Gate attribution	ฮิสโทแกรมรหัสเหตุผลของการปฏิเสธต่อชนิด PAI และคะแนนของการโจมตีที่ผ่าน

ขั้นตอนการประเมินภายใน

- ติดตั้งเซิร์ฟเวอร์ประเมินแยกจาก production เปิด `EKYC_RETAIN_FRAMES=all` , `EKYC_FLASH_FRAMES=4` , `EKYC_PULSE_FRAMES=90` , ใช้ backend deepface
- ทุกเซสชันติดป้าย `label` ตามชนิด: `bona_fide` , `print_a4_matte` , `print_cutout` , `replay_phone` , `replay_challenge` , `mask_paper` , `mask_3dprint` , `mask_latex` , `mask_silicone` , `inject_virtual_cam` ฯลฯ โดยผู้โจมตีต้องพยายามผ่านจริง
- อย่างน้อย 30 การนำเสนอต่อชนิดสำหรับการดูครั้งแรก และ 100+ ก่อนอ้างอิงตัวเลข
- รัน `py -3.12 scripts/pad_eval.py <dir> --json eval.json --markdown eval.md` ; ใช้ `--rescore` เพื่อลองเกณฑ์ใหม่บนหลักฐานเดิม
- ลบข้อมูลที่เก็บไว้เมื่อจบรอบ (ต้องได้รับความยินยอมจากผู้ทดสอบทุกคน)

ถ้อยคำที่ใช้ได้: “ประเมินภายในด้วยตัวชี้วัด ISO/IEC 30107-3” พร้อมแนบตาราง · **ถ้อยคำที่ห้ามใช้:** “เป็นไปตาม/ผ่านการรับรอง ISO/IEC 30107-3” จนกว่าห้องปฏิบัติการภายนอกจะประเมิน

รายละเอียดฉบับเต็ม: `docs/pad-evaluation.md` ในซอร์สโค้ด

8. การใช้งานแอปตัวอย่างกับเซิร์ฟเวอร์ทดสอบ

1. ติดตั้ง `ekyc-example-*.apk` จาก GitHub Releases (Android arm64, เซ็นด้วย debug keystore เพื่อทดสอบ)
2. หน้าแรก: ช่อง “Server” มีค่า URL ของเซิร์ฟเวอร์ Railway อยู่แล้ว; วางกุญแจ API ในช่อง “API key” ครั้งเดียว (เก็บใน SecureStore ของเครื่อง)
3. สถานะบรรทัดล่างช่องกรอกต้องแสดง `ok · 0.1.0`
4. “สแกนใบหน้าเพื่อลงทะเบียน” → ทำตามคำสั่ง (มองตรง → 3 คำสั่งสุม → flash สี 4 เฟรม) → ผลลัพธ์และรายชื่อในตาราง “ผู้ที่ลงทะเบียนแล้ว”
5. “ค้นหาว่าเป็นใคร (identify 1:N)” เพื่อทดสอบการจดจำใบหน้าทีลงทะเบียนไว้ ข้อมูลอยู่ใน Postgres ของ Railway และคงอยู่ข้ามการ deploy

ตารางที่ 8 กรณีทดสอบแนะนำ

ลำดับ	กรณี	ผลที่คาดหวัง
1	ลงทะเบียนบุคคลจริง	pass; personId ใหม่ในรายชื่อ
2	identify ด้วยบุคคลเดิม	pass; ชื่อที่ลงทะเบียน; คะแนน match ≥ 0.42
3	identify ด้วยบุคคลอื่น	fail; <code>NO_MATCH</code>
4	ถือภาพถ่าย/โทรศัพท์อีกเครื่องที่มีวิดีโอผู้ใช้	fail; <code>PAD_LOW</code> และ/หรือ <code>FLASH_SPOOF</code>
5	เปลี่ยนบุคคลกลางคัน	fail; <code>IDENTITY_INCONSISTENT</code>
6	ไม่ใส่กุญแจ API	สถานะ “unreachable — Server returned 401”

9. ข้อจำกัดและแผนงานถัดไป

ตารางที่ 9 ข้อจำกัดที่ทราบและแผนงาน (เรียงตามความสำคัญ)

ข้อจำกัด	แผนงาน
rPPG ปรับเทียบบนสัญญาณสังเคราะห์เท่านั้น จึงเป็น advisory	เก็บ <code>bona_fide</code> ≥ 30 คน + <code>mask_*</code> ผ่าน retention แล้ว ดูการกระจายของ <code>pulse.score</code> → ตั้ง <code>pulse_min</code> และ เปลี่ยนเป็น enforce
เซิร์ฟเวอร์ทดสอบใช้ backend onnx จึงไม่มีคำสั่งอำ ปาก/ยิ้ม และ eye rule เป็น advisory	ติดตั้ง backend deepface (image ขนาดใหญ่กว่า) สำหรับการ ทดสอบหน้ากาก
Attestation ตรวจการมีอยู่ของ token เท่านั้น	verify กับ Google Play Integrity / Apple App Attest แล้วเปิด <code>EKYC_REQUIRE_ATTESTATION=true</code>
เกณฑ์การจับคู่ใบหน้าปรับเทียบบน LFW ไม่ใช่กลุ่มผู้ ใช้เป้าหมาย	วัดกับผู้ใช้จริง ≥ 20 คน $\times$ 3 ครั้ง
ยังไม่ได้ทดสอบภาพพิมพ์ ตัดเจาะ และหน้ากากจริง	ตามโปรโตคอลหัวข้อ 7 แล้วจึงติดต่อห้องปฏิบัติการ
น้ำหนัก InsightFace buffalo_l (backend onnx) เป็นสัญญาณอนุญาตเพื่อการวิจัย	ใช้ backend deepface ใน production หรือแก้ไขสัญญาณอนุญาต
ไม่มีการจำกัดอัตราการเรียก (rate limit) และ certificate pinning	ก่อน production

ภาคผนวก ก. รหัสเหตุผลจากเซิร์ฟเวอร์

รหัส	ความหมาย
CHALLENGE_MISMATCH / TIMING_IMPLAUSIBLE / FRAME_MISSING / FRAME_UNREADABLE	โครงสร้างหลักฐานไม่ตรงที่ออกให้ / เวลาไม่สมเหตุสมผล / ภาพไม่ครบ / ถอดรหัสไม่ได้
NO_FACE / MULTIPLE_FACES	ไม่พบใบหน้า / มีบุคคลอื่นขนาดใกล้เคียงในภาพ
QUALITY_SHARPNESS / QUALITY_BRIGHTNESS / QUALITY_FACE_TOO_SMALL	คุณภาพภาพ neutral
PAD_LOW	MiniFASNet ต่ำกว่า 0.45 บนเฟรมคำสั่ง
POSE_NOT_FRONTAL / POSE_INSUFFICIENT_TURN / POSE_SAME_DIRECTION	ท่ามองตรง / หันไม่พอ ($\geq 22^\circ$ จาก neutral) / หันทางเดียวกันทั้งสองครั้ง
EYES_NOT_CLOSED	EAR ไม่ลดลงพอ ($\text{ratio} \leq 0.65$ และ $\text{EAR} \leq 0.12$)
MOUTH_NOT_OPEN / SMILE_ABSENT / EXPRESSION_UNVERIFIABLE	คำสั่งอ้าปาก/ยิ้มไม่ผ่าน / วัดไม่ได้ (fail closed)
IDENTITY_INCONSISTENT	คู่ภาพต่ำกว่า 0.25 (รวม anchor ของ pulse)
FLASH_SPOOF / FLASH_FRAME_MISSING	สีใบหน้าไม่ตามลำดับ flash / เฟรม flash ไม่ครบ
PULSE_ABSENT / PULSE_FRAME_MISSING	ไม่พบชีพจร (เมื่อ enforce) / ภาพช่วง pulse ไม่ครบหรือสั้นเกินไป
FLAT_FACE / FRAMES_DUPLICATE / ATTESTATION_MISSING	planarity (เมื่อ enforce) / เฟรมซ้ำไบต์เดียวกัน / ไม่มี token (เมื่อบังคับ)

NO_MATCH / PERSON_NOT_FOUND	ไม่ตรงกับแม่แบบ (≥ 0.42) / ไม่พบบุคคล
--------------------------------	--

ภาคผนวก ข. ตัวแปรสภาพแวดล้อมที่สำคัญ

ตัวแปร	ค่าเริ่มต้น / ความหมาย
EKYC_BACKEND	deepface (ค่าเริ่มต้น) / onnx (Railway) / fake
EKYC_API_KEYS	ว่าง = ปิด (dev เท่านั้น); ค่าคั่นด้วยจุลภาค
EKYC_FLASH_FRAMES	0 = ปิด; Railway ตั้ง 4
EKYC_PULSE_FRAMES , EKYC_PULSE_DURATION_MS , EKYC_PULSE_RULE , EKYC_PULSE_MIN	0 / 7000 / advisory / 0.5
EKYC_EXPRESSION_CHALLENGES , EKYC_ALWAYS_OPEN_MOUTH , EKYC_EXPRESSION_RULE	true / true / enforce (ใช้ผลเมื่อ backend วัด blendshapes ได้)
EKYC_MOUTH_OPEN_MIN , EKYC_MOUTH_OPEN_DELTA_MIN , EKYC_SMILE_MIN , EKYC_SMILE_DELTA_MIN	0.35 / 0.20 / 0.45 / 0.25
EKYC_PAD_MIN , EKYC_MATCH_MIN , EKYC_CONSISTENCY_MIN	0.45 / 0.42 / 0.25
EKYC_REQUIRE_ATTESTATION	false
EKYC_RETAIN_FRAMES , EKYC_FRAMES_DIR	none (production) / all หรือ on_fail สำหรับ evaluation
EKYC_NEUTRAL_YAW_MAX_DEG	25 (deepface) / 45 (onnx)

ผู้จัดทำ

ผู้ตรวจสอบ

ผู้อนุมัติ